

DON'T GO BROKE IN THE AGE OF AI

Before the Agent Starts

[COVER ILLUSTRATION — style-match required]

Where This Booklet Fits: This is booklet 3 of 5 in the Don't Go Broke series.

Disclaimer: This booklet provides general educational information and is not formal financial, legal, or professional engineering advice.

Table of Contents

- Field Note Opening
- Section 1 – What You Need Before This Booklet
- Section 2 – The Agent Is Not Just Typing Suggestions
- Section 3 – The Sandbox Strategy
- Section 4 – The Agent Readiness Audit
- Section 5 – Agent Scope Levels
- Section 6 – The Agent Permission Ladder
- Section 7 – Decide What the Agent May Change
- Section 8 – Decide What the Agent Must Not Touch
- Section 9 – Safe vs. Unsafe Agent Requests
- Section 10 – Define the Safe First Task
- Section 11 – Set Tool and Command Boundaries
- Section 12 – Make the Agent Stop and Explain
- Section 13 – What to Do Before Saying "Yes"
- Section 14 – When to Stop Using the Agent
- Section 15 – Reviewing the Agent's Change Receipt
- Section 16 – The Agent Session Log
- Section 17 – Before / During / After: The Operating Rhythm
- Section 18 – The Bad Agent Session: A Recovery Walkthrough
- Field Note Closing
- Back Matter

Field Note Opening

In the previous booklet, we established exactly what you are building. You have a plan, a clean technology stack, and a clear set of user flows. The foundation is firmly set.

Now, you are moving from planning into execution. You are staring at an empty chat box connected to an AI coding agent.

Your instinct is to type: *"Build the dashboard."*

Please, pause before you do.

The moment before you hit "Enter" on a broad task is often the most dangerous — and expensive — moment in your build.

If you are using an autonomous coding agent, you are no longer just receiving code suggestions. You have handed the keyboard to an incredibly fast, highly capable junior developer who never sleeps, never hesitates, and — crucially — never asks for permission unless you force it to.

Before the agent starts acting, you must define its boundaries. A brilliant blueprint won't save your project if the agent is allowed to roam freely and rewrite working code as it pleases. You are still in charge, and you must tell the agent what it is allowed to touch, what tools it can use, and exactly when it must stop and wait for your approval.

Speed without direction isn't progress. It's just faster chaos. This booklet is about putting up the guardrails before you step on the gas.

Section 1

What You Need Before This Booklet

This booklet assumes you are moving past the initial planning phase and are ready to let an agent touch real files. Before you proceed, you should have a few basics in place:

- **A project folder (repository):** A dedicated place where your code lives.
- **A way to save and roll back:** A system (like Git) to snapshot your work so you can safely undo mistakes.
- **A basic understanding of agent capabilities:** Knowing that agents can change multiple files and run terminal commands autonomously.

A note on version control: If you do not use Git or a similar version control tool yet, please pause. Do not let an autonomous agent edit your files until you have a reliable way to make a safe copy. You do not need to be a version control expert, but you must know how to save a working state and roll back when things break. If you aren't there yet, use an agent in "explain-only" mode to help you set up basic version control before proceeding.

None of this needs to be elaborate. A single GitHub repository and the habit of committing before every agent session is enough to start. What matters is that the habit exists before the first prompt, not after the first mistake. Builders who skip this step almost always regret it during their very first agent session, not their tenth.

If you are still deciding on a workflow, the specific tool matters far less than the habit itself. Git, a cloud-synced folder with version history, or even a disciplined system of dated zip files all work, as long as you can always answer one question: what did this project look like five minutes ago, before the agent touched it?

Section 2

The Agent Is Not Just Typing Suggestions

If you have used ChatGPT to write a Python script, or if you use GitHub Copilot to autocomplete a function, you are used to passive AI.

Passive AI types where your cursor is. It waits for you to ask a question. It gives you a block of code, and you decide where to paste it. If it gives you bad code, nothing breaks until you manually integrate it into your project.

Agentic coding tools are fundamentally different. They are autonomous.

An agent can read your entire codebase. It can open files, delete lines of code, and create new directories. It can run commands in your terminal. If it encounters an error, it doesn't wait for you to fix it; it will read the error log and try to fix the error itself.

The preparation required for a passive tool is completely different from the management required for an active agent. You are not managing a spellchecker. You are managing a worker. And if you give a fast worker a vague instruction, they will guess what you meant.

When an agent guesses, it usually guesses wrong. It will over-engineer simple solutions, install heavy libraries you don't need, and rewrite working code because it thought it was "cleaning up."

Field Note: The Danger of the Helpful Agent

A builder asked an agent to "make the login button blue." The agent noticed that the CSS was written in raw CSS, not Tailwind. It decided Tailwind was a "best practice." It installed Tailwind via the terminal, rewrote all the CSS files, updated the build scripts, and broke the entire application. The button was blue, but nothing else worked.

Section 3

The Sandbox Strategy

Because agents have the power to execute changes across multiple files simultaneously, you must never test a new, untethered agent directly on your production-ready codebase.

Before the agent starts, you need a Sandbox.

A Sandbox is an isolated environment where the agent can make mistakes, break things, and hallucinate dependencies without causing permanent damage to your core project.

How to Create a Sandbox:

1. **Git Branches:** The simplest sandbox is a new Git branch. Never let an agent operate directly on `main` or `master`. Create a branch called `agent-experiment-dashboard`. If the agent ruins the code, you simply delete the branch and start over.
2. **Dummy Repositories:** If you are asking the agent to establish a completely new architectural pattern (like setting up a complex database ORM), have it do the work in a completely separate, empty project folder. Once the agent proves it can build the architecture correctly, you can manually move the successful code into your real project.

If you don't yet have version control set up, please pause and establish a "Save Point" system before letting the agent write code. You must have a reliable way to roll back when the agent makes a catastrophic mistake.

[ILLUSTRATION — style-match required]

Section 4

The Agent Readiness Audit

Before letting an agent touch your project — even in a sandbox — you must perform a practical audit of your current state. Handing an agent a messy, broken project and asking it to build a new feature guarantees failure. The agent will get confused by existing bugs and try to fix them instead of doing the assigned work.

Before typing your prompt, fill out this mental (or physical) audit:

1. Project State Verification

- **Is the project currently running?** (If the app currently crashes on boot, stop. Do not ask an agent to build a new feature. Your only prompt should be "Help me debug this crash.")
- **Are there uncommitted changes?** Run `git status`. If you have uncommitted files, commit or stash them immediately. Never let an agent touch a working directory that has uncommitted human work. If the agent makes a mistake, you won't be able to easily separate its bad code from your good code.

2. Scope Targeting

- **Files likely to be touched:** Can you explicitly list the 2 or 3 files this feature *should* require changing?
- **Risky files exposed:** Are there core files (like `server.js`, `database.config`, or `App.jsx`) that the agent absolutely should not touch during this task?
- **Dependencies:** Does this task actually require a new package, or can it be done with existing libraries? (Default to: No new dependencies allowed).

3. Safety Boundaries

- **Commands likely to be run:** Will the agent need to run migrations or just start the dev server?
- **Secrets / Environment files:** Is your `.env` file safely ignored in `.gitignore`? Does the agent have any reason to ask for API keys? (If yes, create mock keys for the agent).

4. The Exit Strategy

- **Rollback plan:** If the agent destroys the next hour of work, how exactly will you revert? (e.g., `git reset --hard HEAD`).
- **Stopping point:** What is the precise, visual proof that the task is complete? When will you tell the agent to stop?

If you cannot answer these questions, you are not ready to deploy an agent. You need to do more architectural planning first.

Why Skipping This Audit Is So Tempting

The audit takes ten minutes. Skipping it takes zero. That asymmetry is exactly why builders skip it — right up until the moment an agent mixes its own broken code into an already-broken project, and nobody can tell which bug came from where.

Here is what that actually looks like. A founder hands an agent a project with a known, unrelated bug in the checkout flow and asks for a new profile page. The agent, reading the codebase, notices the checkout bug along the way and "helpfully" tries to fix it too — without being asked. Now there are two sets of changes tangled together: the profile feature you wanted, and an unrequested checkout fix you never reviewed. If either one breaks, you have no clean way to isolate which change caused it.

The audit is not bureaucracy. It is the ten minutes that keeps every future problem traceable to a single, isolated cause.

Treat the audit as a fixed cost you pay once per session, not once per project. A project that passed the audit yesterday can fail it today if you forgot to commit last night's manual edits. Re-run the checklist every time you are about to hand the keyboard to an agent, not just the first time.

Section 5

Agent Scope Levels

Not all agent tasks carry the same risk. You must actively manage the level of authority you grant the agent based on the complexity of the task and your own understanding of the codebase.

Think of agent authority in seven distinct levels. You should always aim to use the lowest effective level of scope.

Level 1: Explain-Only (Zero Risk) The agent is only allowed to read files and explain how they work. It cannot suggest changes. *Safe for:* Beginners trying to understand a new codebase.

Level 2: Plan-Only (Very Low Risk) The agent reads files and outputs a plain-English plan of how it *would* implement a feature, but it is forbidden from writing the actual code. *Safe for:* Planning complex features before executing them.

Level 3: Single-File Edit (Low Risk) The agent is explicitly restricted to modifying one specific file (e.g., `Button.css`). *Safe for:* Styling adjustments, fixing isolated bugs, or updating static content.

Level 4: Multi-File Edit (Medium Risk) The agent modifies several related files (e.g., a React component, its CSS module, and its test file). *Safe for:* Building standard UI components where the boundaries are clearly defined. Requires careful Git diff review.

Level 5: Test/Run Only (Medium Risk) The agent is authorized to write code and execute terminal commands to run test suites or build scripts to verify its work. *Safe for:* TDD (Test-Driven Development) workflows. *Risk:* The agent might get stuck in an infinite loop of failing tests and rewriting code.

Level 6: Dependency Change (High Risk) The agent is authorized to modify `package.json`, run `npm install`, or update system dependencies. *Safe for:* ONLY when explicitly supervised. *Risk:* The agent installs bloated, deprecated, or malicious packages. Requires strict manual approval.

Level 7: Architecture Change (Critical Risk) The agent is permitted to rewrite core routing, database schemas, or authentication flows. *Safe for:* Advanced builders working in a highly isolated Sandbox. *Risk:* The agent fundamentally alters the application's behavior in ways the human operator no longer understands.

Section 6

The Agent Permission Ladder

If you are new to working with AI agents, you must not start at Level 7. You must climb the Agent Permission Ladder. This ladder teaches you to grant permissions gradually, building trust and verification habits along the way.

Step 1: Ask Questions Only Start by pasting your blueprint or referring to a file and asking: *"Explain how this file works."* or *"What files would I need to touch to add a user profile?"* Do not ask for code. Wait for the explanation. Validate that the agent actually understands your project's architecture.

Step 2: Produce a Plan Once the agent proves it understands the context, ask it to write a plan. *"Write a step-by-step plan for how we will add the user profile. Do not write the code yet."* Review the plan. Does it make sense? Does it touch the right files?

Step 3: Suggest File Changes Ask the agent to output the specific code changes it wants to make. You manually copy and paste those changes into your editor. This forces you to read every line the agent produces.

Step 4: Edit One Safe File Grant the agent permission to autonomously edit a single, non-critical file. *"You may now implement step 1 of your plan by editing `ProfileHeader.jsx`. Do not touch any other files."* Review the diff in your version control tool.

Step 5: Edit Related Files Allow the agent to edit a small cluster of files. *"Implement the profile form logic across `ProfileForm.jsx` and `api/profile.js`."*

Step 6: Run Local Commands Finally, once you trust the agent's code generation and you have a solid rollback plan, allow it to run local test or build commands to verify its own work.

Never skip straight to Step 6. If you give an agent full autonomy immediately, it will optimize for speed, not safety, and it will leave you with a codebase you do not understand.

[ILLUSTRATION — style-match required]

Section 7

Decide What the Agent May Change

Because an agent can touch anything in your project, your first job is to tell it exactly what it is allowed to touch today.

If you ask a human contractor to paint your kitchen, you don't want to come home and find they also demolished your bathroom because they thought the tile looked outdated. Agents will do exactly this to your code.

You must define the allowed scope in plain language before every major task.

Vague: *"Style the login button."*

Bounded: *"You may only read `src/components/Login.js` and edit `src/css/styles.css` to style the login button."*

Restrict the agent's playing field to only what is strictly necessary for the immediate task. If you are building a new UI button, the agent has no business reading your database schema or editing your routing logic.

Scope Statements Are Cheap Insurance

Writing "you may only touch these two files" takes ten seconds longer than just typing your request. That ten seconds is the cheapest insurance policy in this entire process.

Without it, the agent is not being reckless — it is doing exactly what it was built to do: solve the problem in front of it using whatever context is available, including files you never intended it to consider. A scope statement removes the ambiguity before it has a chance to turn into an unwanted rewrite.

Write it down every time, even for tasks that feel too small to bother.

Section 8

Decide What the Agent Must Not Touch

Defining what the agent *can* touch is only half the battle. You must explicitly protect your core assets.

Every project has critical load-bearing code. It might be your authentication logic, your payment processing integration, or a complicated database query that finally works after three days of debugging.

If you don't understand how a piece of working code works yet, the agent should not be allowed to rewrite it.

Introduce "do not touch" boundaries early in your prompt.

Example Boundary: *"Do not modify the `auth/` directory under any circumstances. If a feature requires changing how authentication works, stop and ask me first."*

Agents love to "clean up" code. They will see a working configuration file, decide it isn't using the latest syntax, and rewrite it. Suddenly, your app won't deploy, and you have no idea why. Protect your working parts.

The .gitignore Defense

If you have sensitive API keys, `.env` files, or proprietary data in your project folder, do not rely purely on a text prompt to stop the agent from reading them. Ensure your `.gitignore` is properly configured. Many agents respect `.gitignore` files and will physically prevent themselves from indexing or reading ignored files.

Section 9

Safe vs. Unsafe Agent Requests

First-time builders often treat agents like magic wands. They drop massive, multi-step requests into the prompt window and expect a finished product.

These are not tasks; they are wishes. And when an agent is given a wish, it will try to fulfill it all at once, opening a dozen files, installing random libraries, and tangling the logic into an unrecoverable knot.

To use an agent safely, you must learn to translate vague wishes into concrete, bounded requests.

Unsafe Request: *"Fix the app."* *Why it's dangerous:* The agent will guess what is broken. It will run test commands, read every error log it can find, and start rewriting core files to make the errors go away. **Safe Request:** *"When I click the submit button on `Contact.jsx`, I get a 500 error in the console. Read `Contact.jsx` and `api/submit.js` to identify why the payload is failing. Do not write a fix yet, just explain the error."*

Unsafe Request: *"Make login work."* *Why it's dangerous:* The agent will attempt to build an entire authentication system from scratch, likely installing complex libraries like Passport.js or NextAuth without asking, heavily modifying your database schema in the process. **Safe Request:** *"Create the UI components for a login form in `Login.jsx` with email and password fields. Do not write the backend authentication logic. Do not install any auth libraries. Just build the visual shell and `console.log` the form submission."*

Unsafe Request: *"Clean up the project."* *Why it's dangerous:* "Clean" means nothing to an AI. It will arbitrarily enforce linting rules, reorganize your folder structure, rename variables, and break import paths across your entire application. **Safe Request:** *"Review `utils/helpers.js`. Identify any unused functions and remove them. Ensure the remaining functions have standard JSDoc comments. Do not touch any other file."*

Unsafe Request: *"Improve the UI."* *Why it's dangerous:* The agent will likely inject a massive UI library (like Material UI or Bootstrap) and begin rewriting every component in your project to use that library's specific syntax. **Safe Request:** *"Update the CSS in `sidebar.css` to use a darker color palette. Change the*

background to dark gray and the text to white. Ensure hover states remain visible. Do not change the HTML structure."

Unsafe Request: *"Add payments."* *Why it's dangerous:* The agent will attempt a full Stripe integration, modifying backend routes, database webhooks, and frontend checkout flows simultaneously. If it fails halfway through, your codebase is permanently scarred. **Safe Request:** *"Review the Stripe API documentation for creating a checkout session. Write a plain-English plan for how we will integrate this into `checkout.js` and `server.js`. List the exact steps and wait for my approval."*

Unsafe Request: *"Optimize performance."* *Why it's dangerous:* The agent might implement aggressive caching, rewrite your database queries, or attempt to implement server-side rendering, fundamentally altering the architecture of your application. **Safe Request:** *"The image loading on `Gallery.jsx` is slow. Review the component and suggest three ways we can implement lazy loading. Do not change the code yet."*

The Pattern Behind Every Safe Request

Notice what every "Safe Request" above has in common: it names the exact file, describes the exact symptom, and explicitly forbids the exact shortcut the agent is most likely to reach for. None of them trust the agent to infer what you meant.

This is not extra caution for its own sake. It is a direct response to how these tools actually fail — not through malice, but through an eagerness to be maximally helpful with whatever authority you leave undefined. The fix is always the same shape: narrow the request until there is only one reasonable way to fulfill it.

This is a skill you build with repetition, not a rule you memorize once. The more Safe Requests you write, the faster narrowing a vague idea into a bounded one becomes — until it happens automatically, before you even finish typing the prompt.

[ILLUSTRATION — style-match required]

Section 10

Define the Safe First Task

As demonstrated above, you must break your blueprint down into a **Safe First Task**.

A safe first task is narrow, isolated, and highly verifiable. It has a clear definition of "done."

Start with a small, testable chunk of work. Verify it works. Then give the agent the next small piece. Never ask an agent to perform more than one logical leap at a time. If a task requires the agent to think about UI, database storage, and email notifications all at once, the task is too large. Split it into three tasks.

What a Safe First Task Looks Like

Compare two ways of starting the same feature. The unsafe version: "Build the user profile page." The safe version: "Create a `ProfilePage.jsx` component that displays the user's name and email from the existing `useUser()` hook. No editing, no image upload yet — just a read-only display. Do not touch any other files."

The second version has a visible finish line. You will know within minutes whether it worked, because there is only one thing to check. The first version could take an agent in a dozen directions — styling choices, data fetching, form validation — none of which you asked for and all of which you now have to review.

Small, verifiable steps feel slower at the start. They are actually the fastest path through a project, because every step either clearly works or clearly fails — there is no murky middle ground to untangle later.

When in doubt, make the task smaller, not larger. You can always chain another small task right after a clean success.

Section 11

Set Tool and Command Boundaries

Agents are increasingly capable of running commands directly in your terminal. This is a massive productivity boost, but it is also the fastest way to break your computer.

An agent can run `npm install` to add new dependencies. It can run database migrations. It can push code to production. It can delete directories.

You must set hard boundaries around what commands the agent is allowed to run without your explicit, manual approval.

The Dangerous Command List

You must explicitly ban the agent from autonomously executing the following types of commands. If an agent wants to run these, it must stop and ask for human permission.

1. Package Installation Commands (`npm install`, `pip install`, `yarn add`) The most common and destructive silent action an agent takes is installing unneeded dependencies. If an agent doesn't know how to write a sorting algorithm, it will install a third-party library to do it. Over time, your simple app will become bloated with dozens of unmaintained dependencies, creating security vulnerabilities and deployment nightmares.

2. Deletion Commands (`rm -rf`, `drop table`) Agents should never be permitted to execute blanket delete commands or drop database tables. If an agent wants to remove files, it should list them and wait for you to delete them, or explicitly request permission.

3. Database Migration Commands (`npx prisma db push`, `knex migrate:latest`) Altering a database schema is a permanent, high-risk action. If the agent makes a mistake in a migration, it can destroy your data or break the backend completely. Migrations must always be reviewed by a human before execution.

4. Deployment Commands (`npm run deploy`, `git push origin main`) An agent must never deploy code to a live production environment. The agent's jurisdiction ends at your local development environment.

5. Global Formatting Commands (`eslint --fix .`, `prettier --write .`) While formatting sounds safe, running a global auto-fix command can alter hundreds of files simultaneously. If the agent's formatting rules differ from yours, your Git history will become an unreadable mess of whitespace changes, masking the actual logic changes the agent made.

6. System Configuration Commands (modifying `~/.bashrc`, changing permissions) Agents should operate strictly within the boundaries of your project folder. They should never execute commands that alter your operating system's configuration, user permissions, or global environment variables.

Dependency Change Review Checklist

If the agent pauses and asks for permission to install a new dependency, do not blindly approve it. Ask yourself (and the agent) these questions first:

- ☐ What specific package is being added?
- ☐ Why is it needed? (Can we write the 10 lines of code ourselves instead?)
- ☐ Is there already an existing package in our project that solves this?
- ☐ Does this package affect production deployment?
- ☐ Does it change build behavior or significantly increase the bundle size?
- ☐ Does it create an ongoing maintenance or security cost?

If you cannot answer these questions, reject the dependency and ask the agent for an alternative solution that uses standard code.

Section 12

Make the Agent Stop and Explain

Agents will make silent architectural decisions unless you stop them.

If you tell an agent to "save user preferences," it has to decide *how* to save them. Should it use local storage? A database? A flat file? If you haven't told it how, it will choose for you. It might choose a complex, expensive database solution when all you needed was a simple text file.

You dictate the workflow rhythm. The agent stops when you say stop.

The most effective boundary you can set is an approval gate. Before the agent writes a single line of code, require it to explain its plan.

It is infinitely easier to correct a bad plan written in plain English than it is to untangle bad code spread across five different files.

What "Stop and Explain" Actually Prevents

Every silent decision an agent makes on your behalf is a decision you did not review. Most of the time, that decision is harmless. Occasionally, it is the one that costs you a weekend.

The approval gate is not about slowing the agent down for its own sake. It is about moving the moment of decision from after the code exists to before it exists — when a bad plan is just a paragraph you can delete, not a tangle of files you have to untangle. A plan you can read in thirty seconds is always cheaper to fix than the code it would have produced.

This is especially true early in a project, before you have built the instinct for what a reasonable plan even looks like. Read every plan fully until that instinct exists.

[ILLUSTRATION — style-match required]

Section 13

What to Do Before Saying "Yes"

When the agent successfully outputs its step-by-step plan, the terminal will often pause with a prompt that says `Proceed? (Y/n)`.

Do not reflexively hit `Y`. This is your approval gate. Before saying yes, perform this rapid review:

1. **Read the Plan:** Did the agent actually address the prompt, or did it misunderstand the request?
2. **Compare Plan to Original Task:** Does the plan expand the scope? If you asked for a button color change, is the agent proposing a redesign of the entire header?
3. **Check the File List:** Look closely at the files the agent intends to modify. Are they only the files you authorized? If the agent plans to touch `auth.js` when building a footer, something is wrong. Reject the plan.
4. **Check the Commands:** Is the agent proposing an `npm install`? If so, why?
5. **Ask What Could Go Wrong:** If the plan involves complex logic, literally type into the chat: *"Before we execute this, what are the edge cases? What could break if this fails?"* Force the agent to critique its own plan.
6. **Confirm Rollback Plan:** Ensure your Git working directory is clean. If you say yes and the agent destroys the code, you need to know you can run `git reset --hard HEAD` and lose nothing.
7. **Decide:**
 - Yes: The plan is tight, bounded, and safe.
 - No: The plan is dangerous. Reject it and rewrite your prompt.
 - *Narrow Scope:* The plan is mostly good, but too large. Tell the agent: *"Only do step 1 for now. Do not do steps 2-5."*

Section 14

When to Stop Using the Agent

There will be moments during a build when the agent becomes a liability rather than an asset. You must recognize when the agent is thrashing and take the keyboard back.

Stop the agent immediately if:

- **The agent keeps expanding scope.** If you ask it to fix a padding issue and it tries to rewrite your CSS variables, stop. It has lost focus.
- **The agent cannot explain its own changes.** If you ask "Why did you delete that function?" and the agent responds with generic apologies or hallucinates a reason, its context window is corrupted. Stop.
- **The agent asks to rewrite core architecture.** If the agent claims it cannot complete a simple task unless it completely replaces your routing system or database schema, stop. It is over-engineering a problem it doesn't understand.
- **The agent touches secrets or deployment config.** If the agent inexplicably opens your `.env` file or modifies your hosting configuration during a routine UI task, cut execution immediately.
- **The agent proposes unexplained package installs.** If the agent tries to install a massive library to solve a trivial problem (like installing `lodash` to sort an array), stop.
- **The agent breaks working behavior unrelated to the task.** If you ask the agent to build a footer, and suddenly your login page stops working, the agent has made unauthorized silent changes. Revert immediately.

When the agent thrashes, do not ask the agent to fix the mess it just made. Revert your code to the last save point, manually review the architecture, and rethink your prompt.

Section 15

Reviewing the Agent's Change Receipt

When the agent claims it has finished a task, the session is not over. Do not immediately move on to the next task. First, you must review the change receipt.

Ask the agent to summarize exactly what it just did, then critically evaluate its response against what you actually asked for.

Look for:

- **What files changed:** Did it touch the specific files you authorized?
- **Why they changed:** Did the logic match the approved plan?
- **What was not changed:** Did it leave your protected files alone?
- **What commands were run:** Did it sneak in a silent installation or migration?
- **What risks remain:** Does the agent mention any edge cases it ignored?
- **What you still need to verify:** What manual testing is required before you can trust this code?

If the change receipt reveals the agent touched files outside the allowed scope, you must treat the entire change as suspect.

Treat this review as non-negotiable, even when the task felt simple. The tasks that quietly go wrong are rarely the ones you expected trouble from — they are the small, "obviously fine" ones nobody thought to double-check. A five-minute read of the change receipt is the difference between catching a problem today and discovering it three features later, buried under code that assumes it was never there.

Five minutes now is always cheaper than an hour later.

[ILLUSTRATION — style-match required]

Section 16

The Agent Session Log

To maintain discipline, treat every complex agent session like a science experiment. Keep a running log of what you ask the agent to do and what the result is. This prevents you from blindly trusting the agent over hours of compounding changes.

Use this simple text template (keep it in a `session-log.md` file in your sandbox):

```
### Session Date/Time:
- **Task Requested:** [e.g., Build the navigation bar]
- **Files Allowed:** [e.g., Header.jsx, header.css]
- **Files Actually Changed:** [Fill in after completion]
- **Commands Run:** [Fill in after completion]
- **Dependencies Changed:** [None / List them]
- **Stop Points Triggered:** [e.g., Agent paused to ask about
mobile layout]
- **Human Decision:** [e.g., Instructed agent to ignore mobile
for now]
- **Result:** [Keep / Revert / Needs Follow-up]
```

By forcing yourself to log the session, you prevent the dangerous habit of chaining ten agent requests together without ever manually verifying the code.

The log does not need to be elegant. A single markdown file that grows messier over time is still doing its job, because its value isn't in how it reads later — it's in the pause it forces before you move to the next request.

Even a single line per session — task, files touched, result — is enough to catch the moment a project's behavior started drifting from what you expected, long before the drift becomes an actual bug report.

Section 17

Before / During / After: The Operating Rhythm

Managing an agent requires a strict operating rhythm. If you abandon this rhythm, the agent will eventually drift out of scope.

Before the Agent Runs

- **Define the task:** Choose one isolated, verifiable step.
- **Define the files:** Explicitly state the allowed file paths.
- **Define the commands:** Explicitly ban installations or destructive terminal commands.
- **Define stop rules:** Require the agent to output a plan before writing code.

During the Agent Run

- **Watch for scope drift:** Monitor the agent's output as it generates code.
- **Pause on unrelated work:** If you see the agent open a file related to authentication when it should be styling a button, press the stop button immediately.
- **Ask for explanations:** If the agent takes an unexpected path, ask "Why did you just open that file?" before allowing it to proceed.

After the Agent Runs

- **Inspect changed files:** Look at the visual diff in your version control tool.
- **Run tests manually:** Never assume the code works just because the agent says it does. Click the buttons. Run the app.
- **Ask what changed:** Generate the change receipt.
- **Commit only when understood:** If you don't understand the code the agent wrote, ask it for line-by-line comments before you commit it to your repository.

Section 18

The Bad Agent Session: A Recovery Walkthrough

Even with perfect boundaries, an agent will occasionally lose context, hallucinate a tool that doesn't exist, or embark on a "Silent Refactor."

The Scenario

You ask the agent to fix the padding on a button. It fixes the padding. It then notices a typo in the header and fixes the typo. It then notices the routing logic is outdated, updates the router, installs a new version of `react-router-dom` via the terminal, breaks the app, and spends the next hour trying to fix the router it just broke.

The agent claims it fixed the padding, but the app no longer boots.

The Recovery Sequence

Do not ask the agent to "fix what you just broke." The agent has lost context. Its mental map of your codebase is now corrupted by the hallucinations and unnecessary changes it just made. Asking it to fix a mess usually results in a deeper mess.

Instead, execute the recovery sequence:

1. **Stop:** Press the halt button in your IDE or terminal. Cut off the agent's execution immediately.
2. **Inspect the Diff:** Look at your Git changes. You will see the button padding change, plus dozens of unauthorized routing changes and a modified `package.json`.
3. **Revert:** Throw away all the agent's changes. Run `git restore .` or `git reset --hard HEAD` to return your project to the last known working Save Point.
4. **Narrow the Scope:** Acknowledge that your initial prompt gave the agent too much leeway.
5. **Restart with Stricter Boundaries:** Start a completely new chat thread. The old thread is poisoned with bad context. In the new thread, explicitly ban the agent from looking at or touching the router.

[ILLUSTRATION — style-match required]

Field Note Closing

The agent should not start until the boundaries are clear.

Taking five minutes to define what the agent cannot do will save you five hours of trying to figure out what it just did.

The goal of setting these boundaries isn't to slow you down. The goal is to keep you in the driver's seat. You are the builder. The agent is just the hands on the keyboard.

None of this is about distrust. Agents are remarkable at what they do, and they will keep getting faster. But speed and judgment are two different things, and only one of them is your job to provide. An agent that moves fast inside boundaries you set is an asset. The same agent moving fast with no boundaries is just risk wearing a helpful face.

The builders who get the most out of these tools are not the ones who hand over the most control. They are the ones who know exactly how much control to hand over, task by task, and adjust as trust is earned.

That adjustment never fully stops. Even on your tenth project with the same agent, the right response to a new kind of task is still to start narrow and expand only once the pattern proves itself. The tool will keep getting more capable. The discipline of deciding how much rope to give it is yours to keep, indefinitely.

Set up your sandbox. Define your Safe First Task. Fill out your checklists.

When you're ready, give the agent its first instruction.

Once you do, the agent will move fast. It will likely generate a first draft of your feature or layout in minutes. The next expensive mistake is trusting that draft too soon. In the next booklet, *Before You Trust the First Draft*, we will cover exactly how to interrogate what the AI produced before you build anything else on top of it. You remain in charge — not just of what gets built, but of what gets kept.

Back Matter

The Agent Prompt Pack

Do not reinvent the wheel every time you open a new chat window. Copy and paste these practical prompts directly into your agent's system instructions or initial request.

The Plan-First Prompt

"STOP AND PLAN RULE: Before you edit any files or write any code, you must output a step-by-step plan. List the exact files you intend to create or modify. List the architectural approach you intend to take. Do not write the actual code or run any commands until I reply with 'Approved'."

The File-Boundary Prompt

"ALLOWED SCOPE: You are only permitted to read and modify the following files: [List files here]. Do not open, read, or modify any other files in this project to complete this task. If you believe you need to touch another file, stop and ask me for permission."

The Command-Boundary Prompt

"COMMAND BOUNDARIES: You are authorized to run local development servers and test suites to verify your work. You are STRICTLY FORBIDDEN from running `npm install`, adding any new dependencies, running database migrations, or executing any destructive commands without asking me first."

The Stop-and-Ask Prompt

"DECISION RULE: If you encounter ambiguity, do not guess. If there are multiple architectural paths to solve a problem, do not pick one silently. Stop, list the options, and ask me which path to take."

The Change Receipt Prompt

"TASK COMPLETE RULE: When you have finished the task, output a Change Receipt. List exactly which files you modified, what commands you ran, and why. List any edge cases or risks that I need to manually verify before committing this code."

The Recovery/Restart Prompt *(Use in a fresh thread after a failure)*

"We are restarting this task because the previous attempt failed due to unauthorized scope expansion. The task is [Task Name]. You are strictly forbidden from modifying [Area that broke previously]. Focus entirely on the isolated task."

The Before-the-Agent Checklist

Before you hit "Enter" on your first agent prompt, confirm your readiness.

- ☐ **Agent Task:** I have defined a narrow, verifiable task.
- ☐ **Allowed Files/Folders:** I have explicitly listed which specific files the agent is allowed to edit.
- ☐ **Off-Limits Files:** I have explicitly protected critical files the agent must never touch.
- ☐ **Command Rules:** I have forbidden the agent from installing new dependencies or running deployments without asking.
- ☐ **Stop Point:** I have required the agent to output a plan before writing code.
- ☐ **Review Requirement:** I know exactly what the agent must show me before it proceeds.
- ☐ **Definition of Done:** I know exactly how I will manually test that this specific task is complete.
- ☐ **Sandbox Ready:** I am on a safe Git branch or working in an isolated environment.
- ☐ **If Unsure:** I have instructed the agent to stop and ask for clarification rather than guessing.

[NEXT BOOKLET COVER — insert when available]